环境与向量执行过程

建立可运行、可观察的整核实验环境，追踪同一条 VADD 从取指到已提交映射的过程。

01 / 环境与执行过程 | 赛题 1.1、1.2、1.3
实验材料截至 2026-10-30 · 2026-11-02 第三版修订

目录

2 环境部署与运行验证

- 2.1 实验平台与仿真流程
- 2.2 工具配置与仿真器构建
- 2.3 Hello 程序与应用加载
- 2.4 运行结果及向量环境检查

3 VADD 指令执行过程

- 3.1 测试程序和波形来源
- 3.2 如何确认追踪的是同一次执行
- 3.3 从取指块中找到 VADD
- 3.4 从逻辑寄存器到可执行操作
- 3.5 计算结果怎样成为架构状态

附录 A 环境与波形索引

2 环境部署与运行验证

后续指令开发需要两种能力：运行真实 RISC-V 程序，并能观察程序在香山整核中的执行过程。第一阶段因此先把源码构建、应用编译、镜像加载和差分检查串成可复现流程，再用定制 Hello 与四元素向量加法确认运行环境。Hello 检查基本启动和输出通路；VADD 则提供可手算的向量结果，并成为下一章波形追踪的对象。

2.1 实验平台与仿真流程

实验在 Windows 11 的 WSL2 Ubuntu 环境中开展，香山使用昆明湖 V2 代码线和单核 MinimalConfig。这里记录的是 2026-10-19 基础环境验收所使用的版本；第三阶段扩展指令时使用的源码 NEMU 另行登记。

表 2-1　基础环境配置。

项目	本次实验配置
Linux 环境	WSL2，Ubuntu 24.04.4 LTS
可用资源	主机约 16 GB 内存、20 个逻辑处理器；WSL 内存约 9.7 GiB、swap 8 GiB
香山源码	16ae9ddc，单核 MinimalConfig，VLEN=128
波形支持	构建时启用 `EMU_TRACE=1`
Java / Mill	OpenJDK 21.0.12.1 / 工程锁定 Mill 0.12.3
RTL 仿真	Verilator 5.052 生成的 `build/emu`
应用工具链	RISC-V GCC 13.3.0、binutils 2.42、Make 4.3
AM 环境	nexus-am 086a2380
第一阶段参考模型	ready-to-run 仓库 ed8c30e1 的预编译 NEMU 共享库

仿真环境由 RTL 仿真器、测试镜像和 NEMU 参考库组成。Chisel/Mill 先生成 RTL，Verilator 再把 RTL 转成 C++ 仿真模型并链接为 `emu`；RISC-V 工具链另将测试程序编译成 ELF 和二进制镜像；运行时，emu 加载镜像，并调用 NEMU 共享库进行差分检查。修改应用的打印或输入只需重建应用，修改硬件才需要重建仿真器。

完整版本及哈希见第一阶段复现索引。用这些版本号和哈希，可以把某次运行结果对应回当时的源码与二进制，也能区分基础环境参考库和后续支持自定义指令的参考库。

2.2 工具配置与仿真器构建

部署过程中先准备 Java、RISC-V 交叉工具链、LLVM/Clang 和 Verilator，再载入 xs-env 工程环境。一次恢复构建在 C++ 编译阶段报出 `clang++: No such file or directory`，退出状态为 Error 127。检查发现编译器安装于 `/usr/local/llvm/bin`，补充 PATH 后继续构建，RTL 生成、Verilator 转换和 C++ 编译均成功结束。

以下是归档记录使用的 WSL/Bash 构建路径和命令：

```
cd /mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env/XiangShan
export PATH="/usr/local/llvm/bin:$PATH"
make emu CONFIG=MinimalConfig EMU_TRACE=1 JVM_XMX=6G -j4
```

CONFIG 选择处理器配置，EMU_TRACE 保留导出波形所需支持，JVM_XMX 和 -j4 分别控制 JVM 堆和编译并行度。编译完成后得到该配置下的仿真器 build/emu。构建使用已有工程的恢复方式，原错误和各阶段退出状态均保留在部署记录〔1〕及构建时间记录〔2〕中。

2.3 Hello 程序与应用加载

定制 Hello 使用 AM 的打印接口，输出个人标识和当次 WSL IPv4，然后正常返回。记录中的地址为 172.26.232.122，它描述该次实验的网络环境。

```
printf("hello xiangshan, I am Meijie, IP address: 172.26.232.122\n");
```

先在 WSL/Bash 编译 AM 应用：

```
export AM_HOME=/home/lenovo1227/xiangshan-learning/nexus-am
cd "$AM_HOME/apps/hello-meijie"
make ARCH=riscv64-xs LINUX_GNU_TOOLCHAIN=1 MARCH=rv64gcv -j2
```

ARCH=riscv64-xs 选择香山运行环境，LINUX_GNU_TOOLCHAIN=1 选择对应工具链前缀，MARCH=rv64gcv 指定应用指令集。编译得到的 .bin 用作仿真器输入，ELF 与反汇编用于定位符号和指令。

下列命令加载本次保存的镜像，显式指定差分参考库：

```
XS_ENV=/mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env
RUN_DIR='/mnt/d/Mayyyyyy/WorkSpaces/日常工作区1/2026比赛规划/RISCV_vdot/artifacts/runs/S1-01/hello-meijie-20261019-
120654-8FtdkQ'
cd "$XS_ENV/XiangShan"
./build/emu \
  -i "$RUN_DIR/hello-meijie-riscv64-xs.bin" \
  --diff "$XS_ENV/XiangShan/ready-to-run/riscv64-nemu-interpreter-so" \
  --no-snapshot -C 100000 -s 1
```

其中 -i 选择应用，--diff 选择参考库，-C 设置最大仿真周期，-s 固定种子。--no-snapshot 关闭快照。运行是否成功同时根据预期输出、DiffTest 状态、GOOD TRAP 和程序退出码判断。

2.4 运行结果及向量环境检查

归档日志和独立退出记录给出以下结果：

```
BUILD_EXIT_CODE=0
Difftest enabled.
hello xiangshan, I am Meijie, IP address: 172.26.232.122
HIT GOOD TRAP at pc = 0x8000014c
EMU_EXIT_CODE=0
```

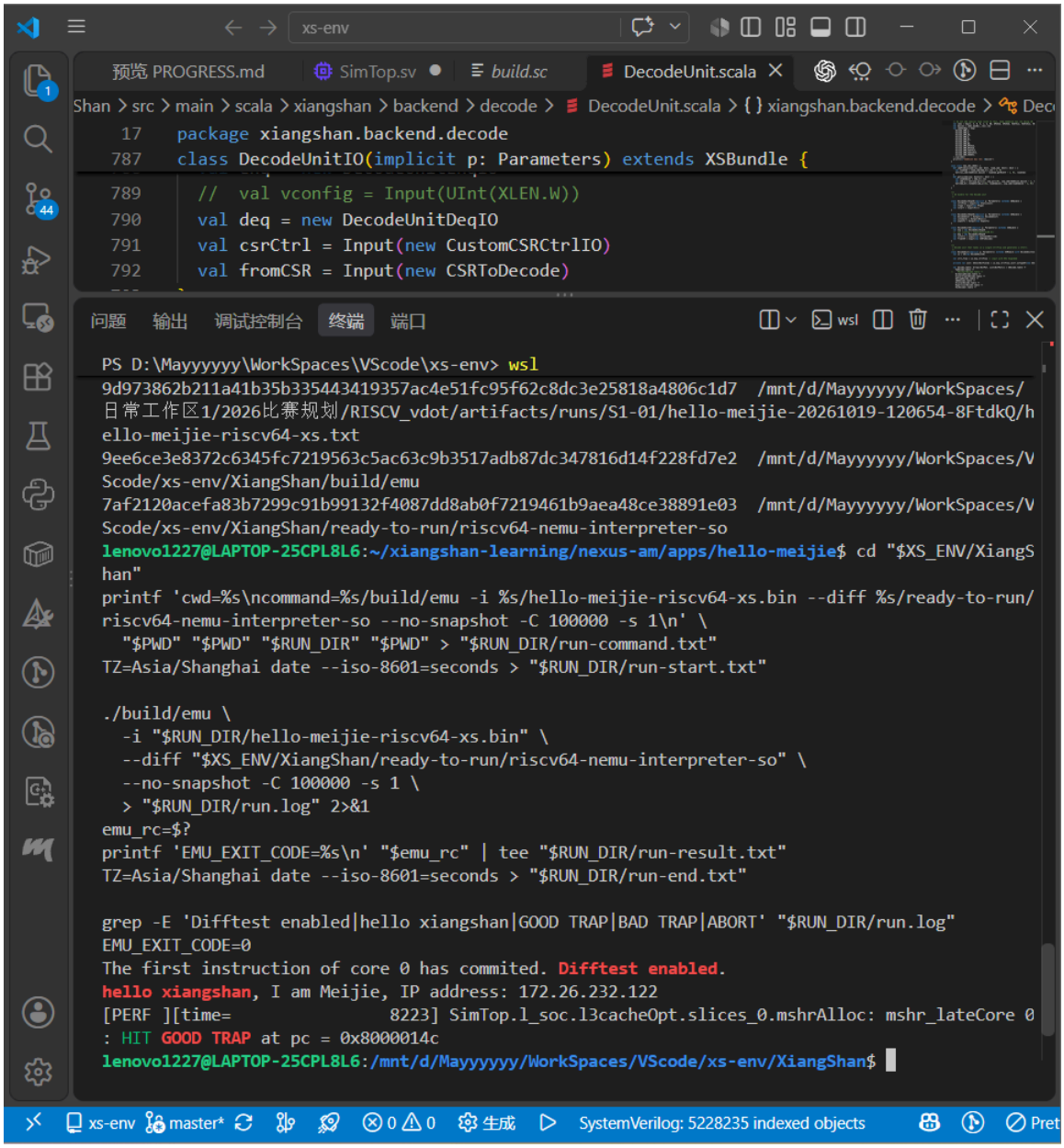


图 2-1　定制 Hello 的运行结果。个人标识、当次 IP、GOOD TRAP 与退出状态共同确认程序完成。

Hello 验证基本启动和输出后，四元素 VADD 将 `[1,2,3,4]` 与 `[10,20,30,40]` 相加，得到 `[11,22,33,44]`。应用逐元素检查期望值，首次运行和导出波形的运行均启用 DiffTest，并正常结束。由此形成两层基础检查：Hello 确认程序能够进入、输出并退出；VADD 确认指定向量配置下的装载、运算和存储路径。

下一章沿同一条 VADD 的动态执行实例，说明取指到提交各阶段如何衔接。相应的 Hello 原始日志〔3〕、退出记录〔4〕和环境核对记录〔5〕列入附录，供复现时核查。

3 VADD 指令执行过程

程序算出正确结果后，还需要解释这条指令如何经过香山前端、乱序后端和提交路径。本章选择一次 `vadd.vv v3,v1,v2` 执行，先确认输入输出，再从取指块追踪到译码、重命名、发射、计算、物理寄存器写入和已提交映射更新。整个分析始终跟踪同一次动态执行，避免把相同 PC 在不同时刻的活动混在一起。

3.1 测试程序和波形来源

测试配置为 VLEN=128、SEW=32、LMUL=1、VL=4，无掩码执行。数组 A=`[1,2,3,4]`、B=`[10,20,30,40]` 分别进入 v1、v2，结果存回 C。核心汇编如下：

```
li t0, 0x200
csrs mstatus, t0
csrwi vstart, 0
vsetvli t1, a3, e32, m1, ta, ma   # a3=4, 检查实际VL
bne t1, a3, .Ldone
vle32.v v1, (a0)
vle32.v v2, (a1)
vadd4_target:
vadd.vv v3, v1, v2
vse32.v v3, (a2)
.Ldone:
mv a0, t1
ret
```

C 程序将输出预填为 −1，并逐项比较独立期望。运行读回 `vlenb=16`，确认 128 位向量宽度，随后给出四项结果和正常结束标志：

```
vadd4: VL=4 SEW=32 LMUL=1 VLEN=128 bits
C[0] = 11 (expected 11)
C[1] = 22 (expected 22)
C[2] = 33 (expected 33)
C[3] = 44 (expected 44)
PASS: vadd.vv produced [11, 22, 33, 44]
Core 0: HIT GOOD TRAP at pc = 0x8000023c
```

```
lenovo1227@LAPTOP-25CPL8L6:/mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env/XiangShan$ grep -E '
Difftest enabled|vadd4:|C\[|PASS|FAIL|GOOD TRAP|BAD TRAP' \
  "$AM_HOME/apps/vadd4/build/run.log"
The first instruction of core 0 has commited. Difftest enabled.
vadd4: VL=4 SEW=32 LMUL=1 VLEN=128 bits
C[0] = 11 (expected 11)
C[1] = 22 (expected 22)
C[2] = 33 (expected 33)
C[3] = 44 (expected 44)
PASS: vadd.vv produced [11, 22, 33, 44]
[PERF ][time=              22954] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc: mshr_late
Core 0: HIT GOOD TRAP at pc = 0x8000023c
lenovo1227@LAPTOP-25CPL8L6:/mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env/XiangShan$
```

图 3-1　VADD 应用结果。结束陷阱位于 `0x8000023c`，本章追踪的加法指令位于 `0x80000148`。

应用和仿真器分别构建。以下命令在第一阶段的 WSL/Bash 环境中编译应用，然后创建新的波形目录运行同一镜像：

```
export AM_HOME=/home/lenovo1227/xiangshan-learning/nexus-am
XS_ENV=/mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env
cd "$AM_HOME/apps/vadd4"
make ARCH=riscv64-xs LINUX_GNU_TOOLCHAIN=1 MARCH=rv64gcv -j2

mkdir -p "$HOME/xiangshan-learning/waves"
wave_dir=$(mktemp -d "$HOME/xiangshan-learning/waves/vadd4-XXXXXX")
cd "$XS_ENV/XiangShan"
./build/emu \
  -i "$AM_HOME/apps/vadd4/build/vadd4-riscv64-xs.bin" \
  --diff "$XS_ENV/XiangShan/ready-to-run/riscv64-nemu-interpreter-so" \
  --no-snapshot --dump-wave-full \
  --wave-path="$wave_dir/vadd4.vcd" \
  -b 0 -e 8000 -C 200000 -s 0 \
  > "$wave_dir/run.log" 2>&1
wave_rc=$?
printf 'EMU_EXIT_CODE=%s\n' "$wave_rc" | tee "$wave_dir/result.txt"
```

`--dump-wave-full` 导出完整层级，`-b/-e` 选择仿真 cycle 窗口，`-C` 限制总周期，`-s 0` 与归档运行的种子一致。首次运行和导波形运行均通过 DiffTest，退出码为 0。复现时可利用生成的 ELF/反汇编定位符号，再在 GTKWave 中载入 VCD；操作入口见原波形指南〔6〕。

3.2 如何确认追踪的是同一次执行

反汇编中的 PC=`0x80000148`、机器码=`0x021101d7` 确定目标静态指令。前端以 FTQ `0/3` 和目标半字偏移 `0xA` 关联取指来源；后端分配后以 ROB `1/16`、物理目的 51 关联本次执行。逻辑源 v2、v1 对应物理源 50、48。

本次 VCD 的时钟周期为 2 ps，偶数 ps 为上升沿。判定 valid/ready 握手时，读取上升沿之前的稳定值，因此许多 Marker 放在事件沿前 1 ps；沿后的数据或 valid 可能已经属于下一条指令。退休则观察 `isCommit && commitValid` 及对应的 ROB/指令标识。这里的 ps 是仿真时间标尺。

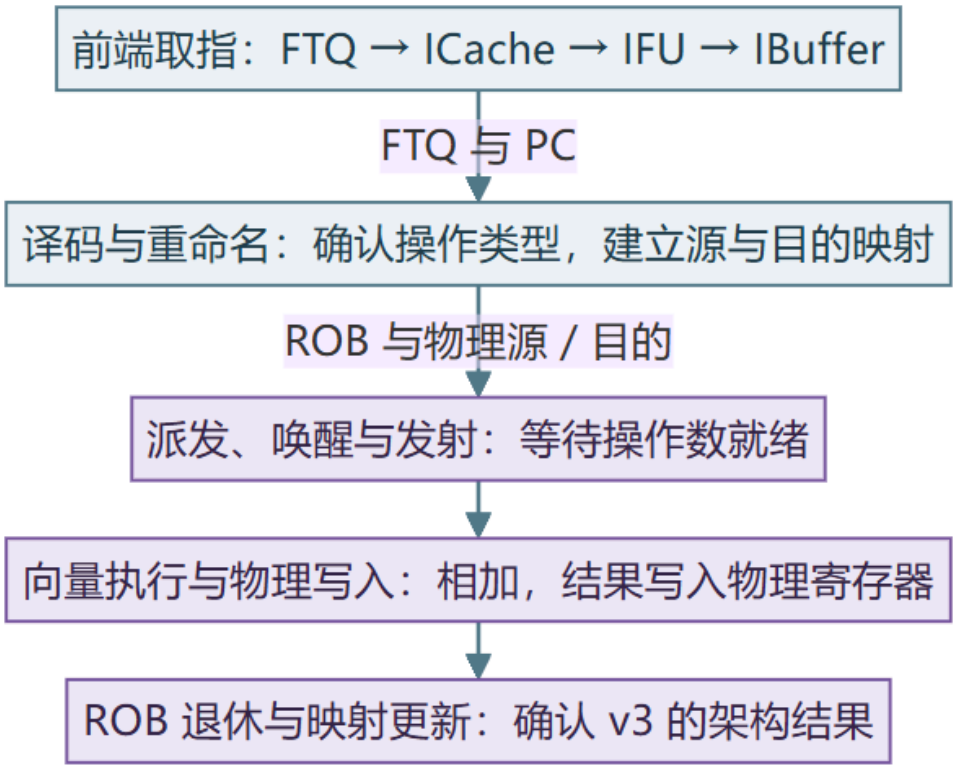


图 3-2　分析路径。前端追踪目标指令所属的取指块，后端追踪操作数就绪、执行、写回和提交过程。

表 3-1 汇总确定的事件沿，以 FTQ 请求接受沿为相对周期零点。它先给出全貌，后文解释各段之间的因果关系。

表 3-1　目标动态指令的主要事件。

事件	VCD 时间	相对周期
FTQ 取指请求接受	5080 ps	0
ICache 应答与 IFU F2 推进	5084 ps	2
IFU 输出进入 IBuffer	5086 ps	3
后端入口接受，目标进入 decodeBuf	5088 ps	4
译码接受目标	5112 ps	16
重命名输出接受	5116 ps	18
派发入队	5118 ps	19
两个物理源先后就绪	5252、5258 ps	86、89
发射接口接受	5262 ps	91
执行单元接受操作数	5268 ps	94
运算输出握手	5270 ps	95
物理寄存器写入	5272 ps	96
ROB 退休	5274 ps	97
v3 已提交映射更新	5278 ps	99

完整信号层级和源码入口见阶段一附录。正文只保留判定事件所需的字段；完整信号层级名称见附录。

3.3 从取指块中找到 VADD

3.3.1 FTQ 请求与 ICache 返回数据

5079 ps 时，FTQ 请求的 valid/ready 均为 1，身份为 `0/3`，起始地址为 `0x80000134`。请求在 5080 ps 被接受，沿后接口立即转向 FTQ `0/4`、地址 `0x80000154`。这说明接口能够连续发送取指块；目标指令位于前一块内部，所以它的 PC 可以不同于该块的起始地址。

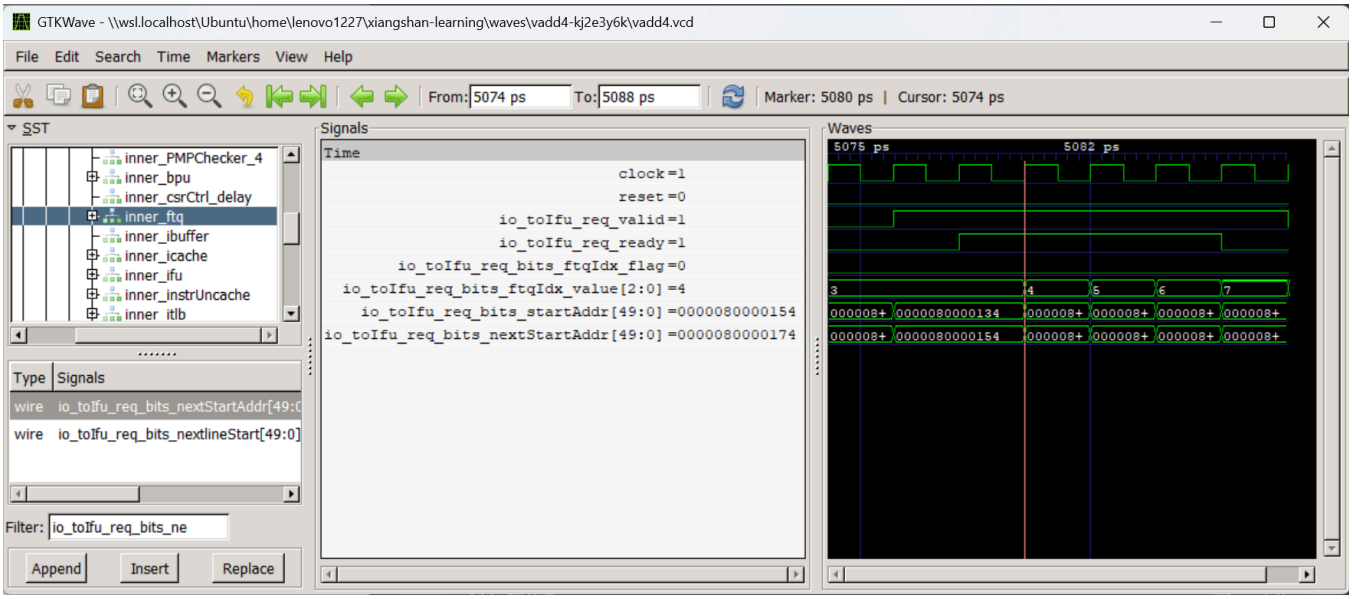


图 3-3　5080 ps 接受 FTQ `0/3` 后，接口转向下一取指块。目标所属请求应读取接受沿之前的 FTQ 编号。

5083 ps，ICache 返回有效应答，地址仍为 `0x80000134`；IFU F2 同时持有 FTQ `0/3`，`f2_fire=1`。本次请求跨越两条缓存行，`doubleline=1`，两侧异常字段均为零。ICache 应答为 Valid 接口，结合对应地址和 IFU 推进条件确认 5084 ps 的交接。

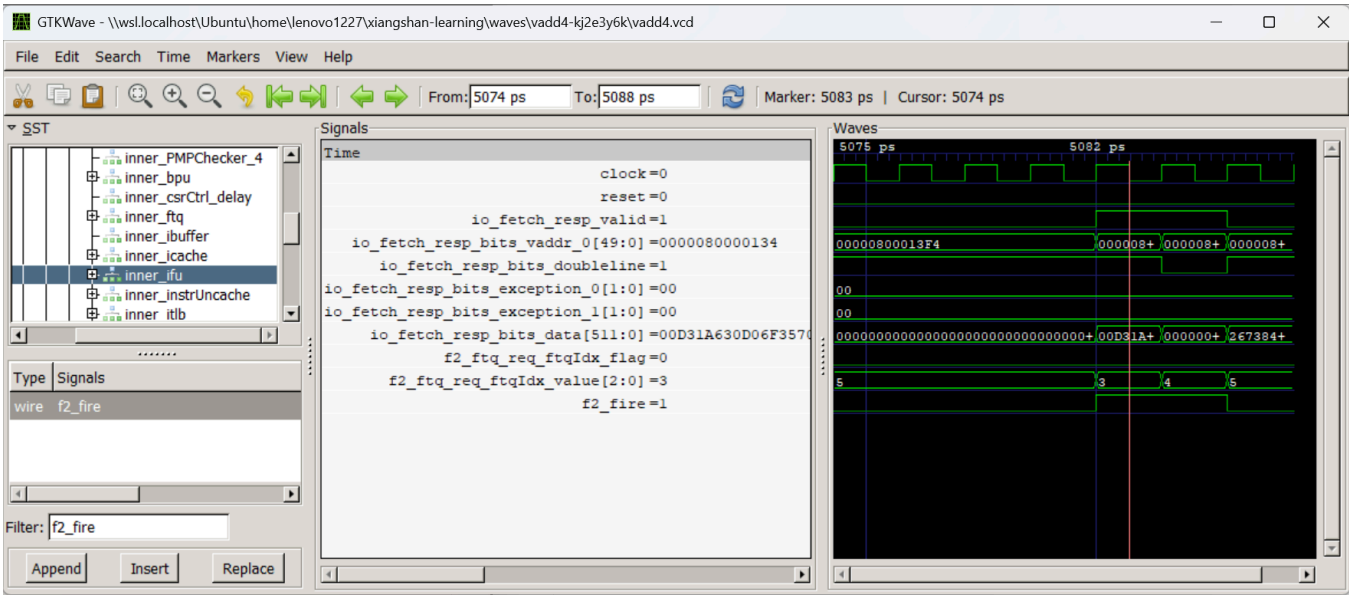


图 3-4　ICache 的有效应答与 IFU F2 中的 FTQ `0/3` 对应。

返回总线最低 128 位为 `020661A7_021101D7_0205E107_02056087`，右端是低位。目标机器码位于 `[95:64]`：

表 3-2　返回数据中的四条连续指令。

位段	地址	机器码与操作
[31:0]	0x80000140	0x02056087，装载 v1
[63:32]	0x80000144	0x0205e107，装载 v2
[95:64]	0x80000148	0x021101d7，目标 VADD
[127:96]	0x8000014c	0x020661a7，存储 v3

请求起点落在前一缓存行末段；返回总线按选中的 8 字节 bank 重新拼装，本例最低部分来自 `0x80000140`。IFU 先对返回数据复制拼接，再用切分指针选出半字：目标对应 `f2_cut_ptr_10/11=36/37`，映射回原数据的半字 4/5。因此，目标在总线中的位置须按切分指针推算，不能把 PC 与请求起点的差值直接当作总线偏移。

3.3.2 IFU 输出和 IBuffer 接收

目标距取指起点 20 字节，即第 10 个半字位置。5085 ps，IFU 位置 10 的 PC、机器码分别为 `0x80000148`、`0x021101d7`，包内 `valid` 和 `enqEnable` 均为 `0xd555`，其 bit10 有效；整包 valid/ready 为 1/1，接收侧没有 flush。目标由此在 5086 ps 进入 IBuffer。

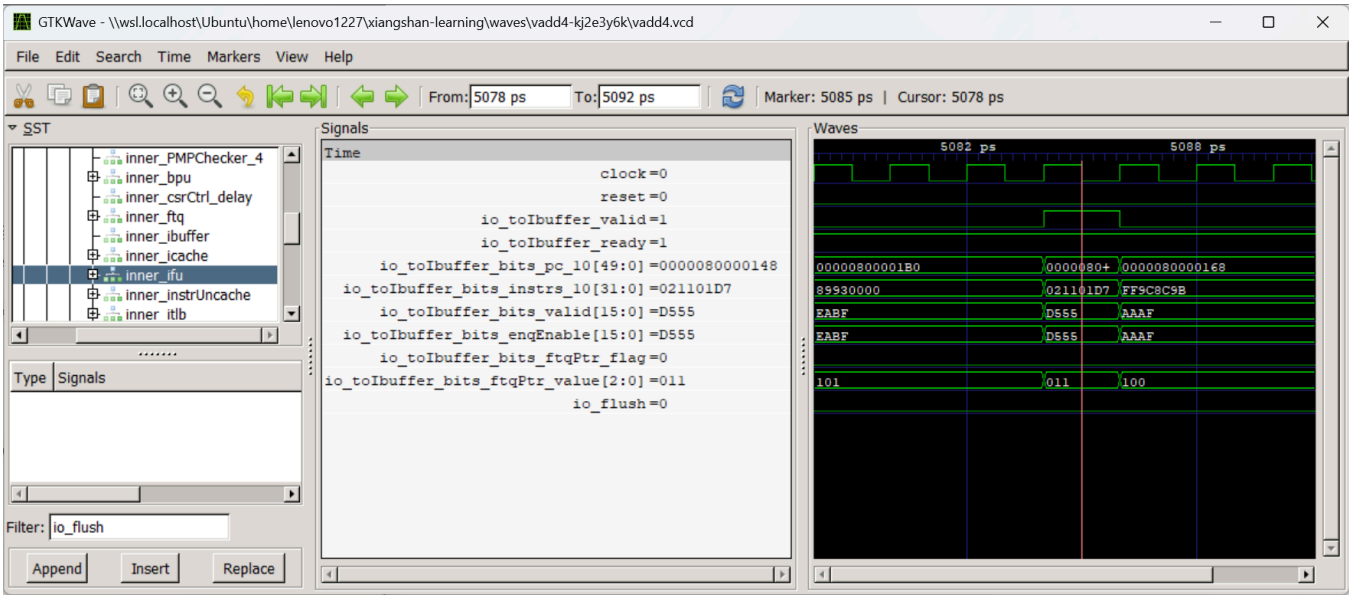


图 3-5　IFU 位置 10 的指令身份、包内掩码和接口握手共同确认接收。

到 5087 ps，目标位于 IBuffer 输出槽 5，FTQ 仍为 `0/3`，`ftqOffset=0xA`。IFU 采用半字位置编号，IBuffer 输出采用指令槽编号，因此从位置 10 变成槽 5，只是编号方式不同，仍是同一条指令。

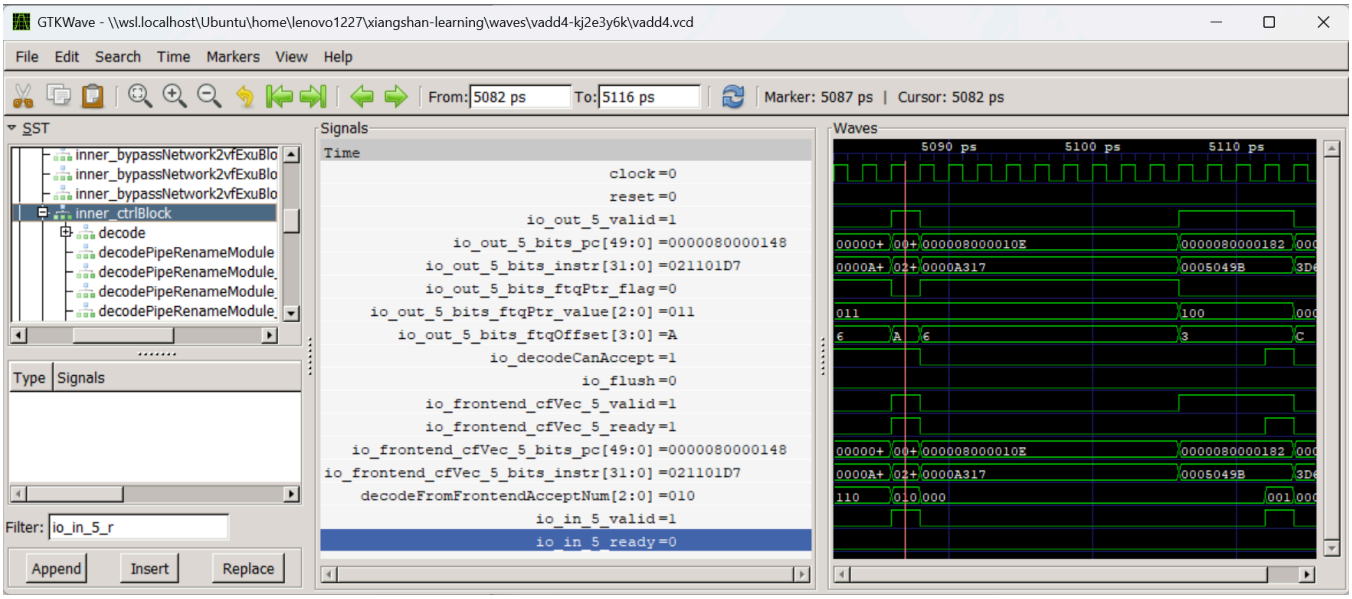


图 3-6　后端入口 `cfVec5` 的 valid/ready 为 1/1，但译码输入 ready 为 0，指令先由入口缓冲保存。

3.3.3 入口接受与译码接受为何相隔十二周期

5088 ps 的后端入口接受并不意味着译码级已经消费目标。当时前两条指令被译码接收，目标从输出槽 5 进入 decodeBuf 槽 3。随后，前方指令继续被消费，目标在 5090、5092 ps 依次移到槽 1、槽 0，PC、机器码和 FTQ 标识保持不变。

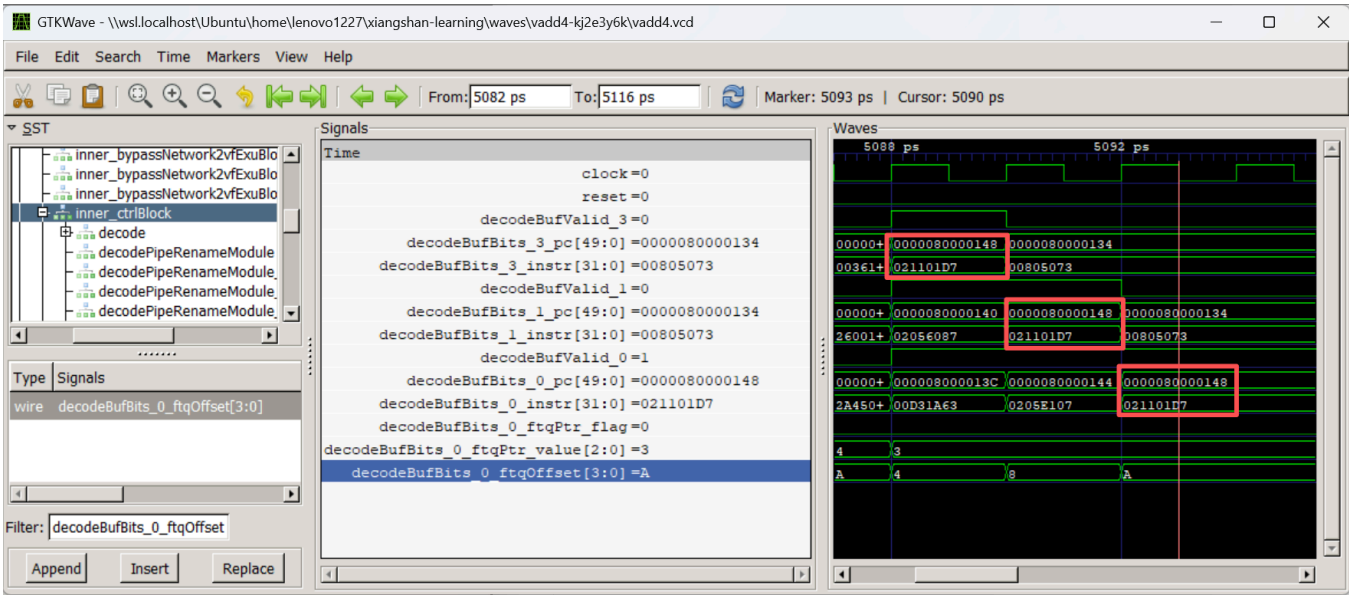


图 3-7　5093 ps 目标位于有效的 decodeBuf 槽 0；这只是缓冲槽位变化，仍是同一条目标指令。

目标到队首后仍需等待译码 ready。5111 ps 的输入 valid/ready 才同时为 1，redirect=0，因此 5112 ps 为真正的译码接受沿。沿后 `decodeBufValid_0` 清零，译码入口转向下一条存储指令 `0x8000014c`。入口 valid 可以继续为 1，因为此时它已经描述下一条指令。

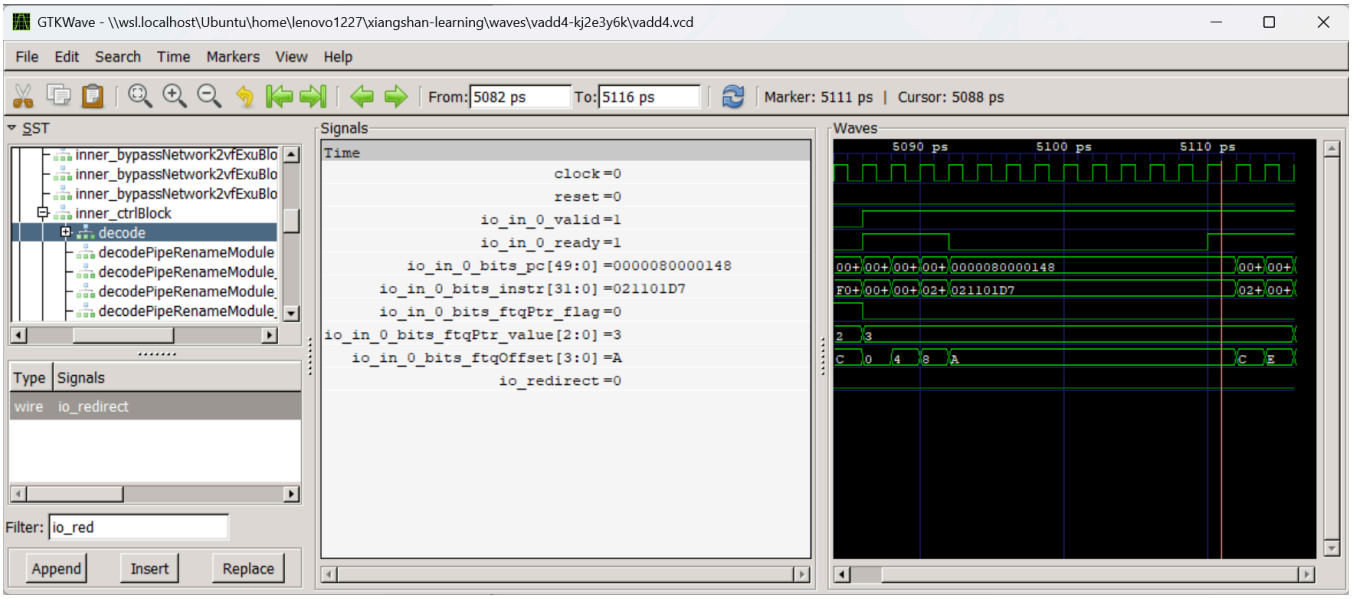


图 3-8　5111 ps 的稳定握手值确定 5112 ps 的译码接受事件。

3.4 从逻辑寄存器到可执行操作

3.4.1 译码建立运算和源类型

译码将机器码解释为向量加法：逻辑源为 v2、v1，逻辑目的为 v3，`vecWen=1`、`rfWen=0`；`fuType=0x80000`、`fuOpType=0x40` 对应本版本的向量整数加法路径。向量控制 `vsew=2`、`vlmul=0`、`vm=1`，即 e32/m1 无掩码操作。

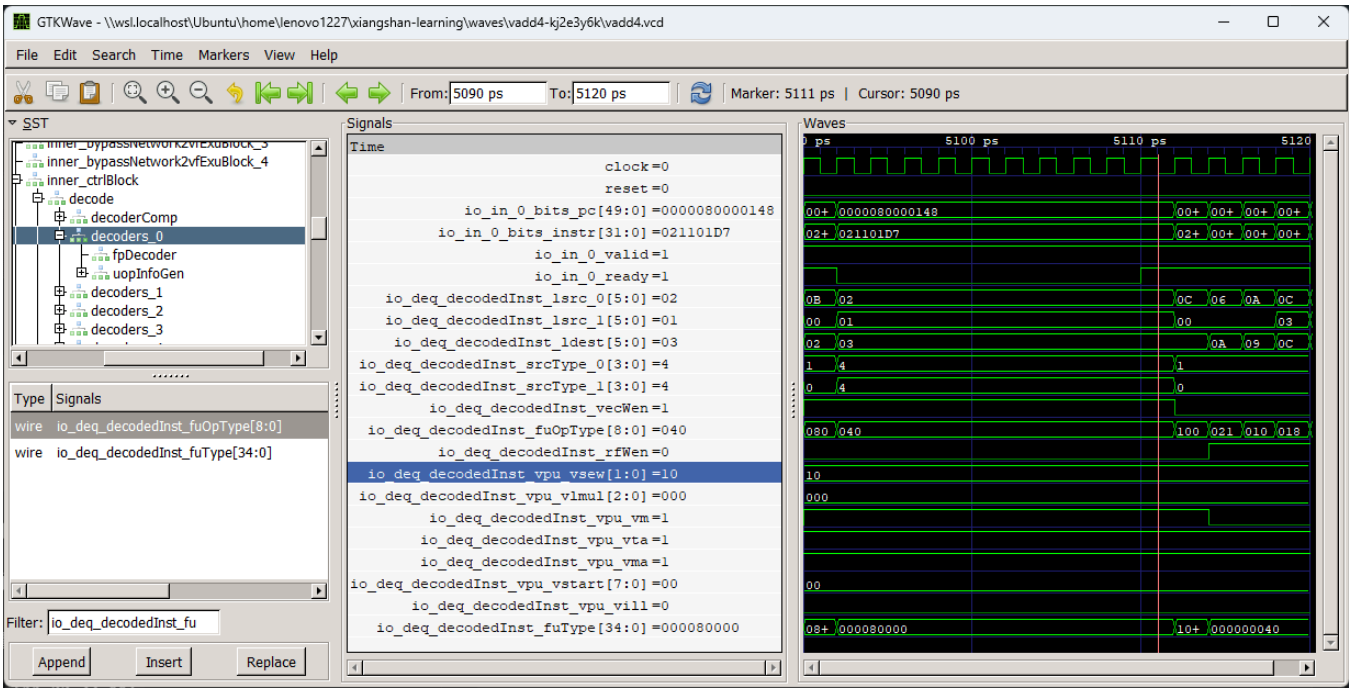


图 3-9　译码把指令编码转换为逻辑源、目的及执行控制。两个源类型均为向量类型。

汇编按 `vd,vs2,vs1` 排列，而本实现的源 0/1 按 RS1/RS2 字段提取，因此该实例的源 0 对应 v2，源 1 对应 v1。通用 old-vd 字段仍能看到逻辑 v3，但本例 `isDependOldVd=0`，旧目的不参与加法。

3.4.2 重命名分配物理版本

5115 ps，重命名输出把 v2、v1 映射到物理源 50、48，为新的 v3 分配物理目的 51，同时携带 ROB `1/16`。valid/ready 为 1/1，输出在 5116 ps 被接受。波形采用十六进制时，这些编号分别显示为 `0x32`、`0x30`、`0x33`，ROB value 为 `0x10`。

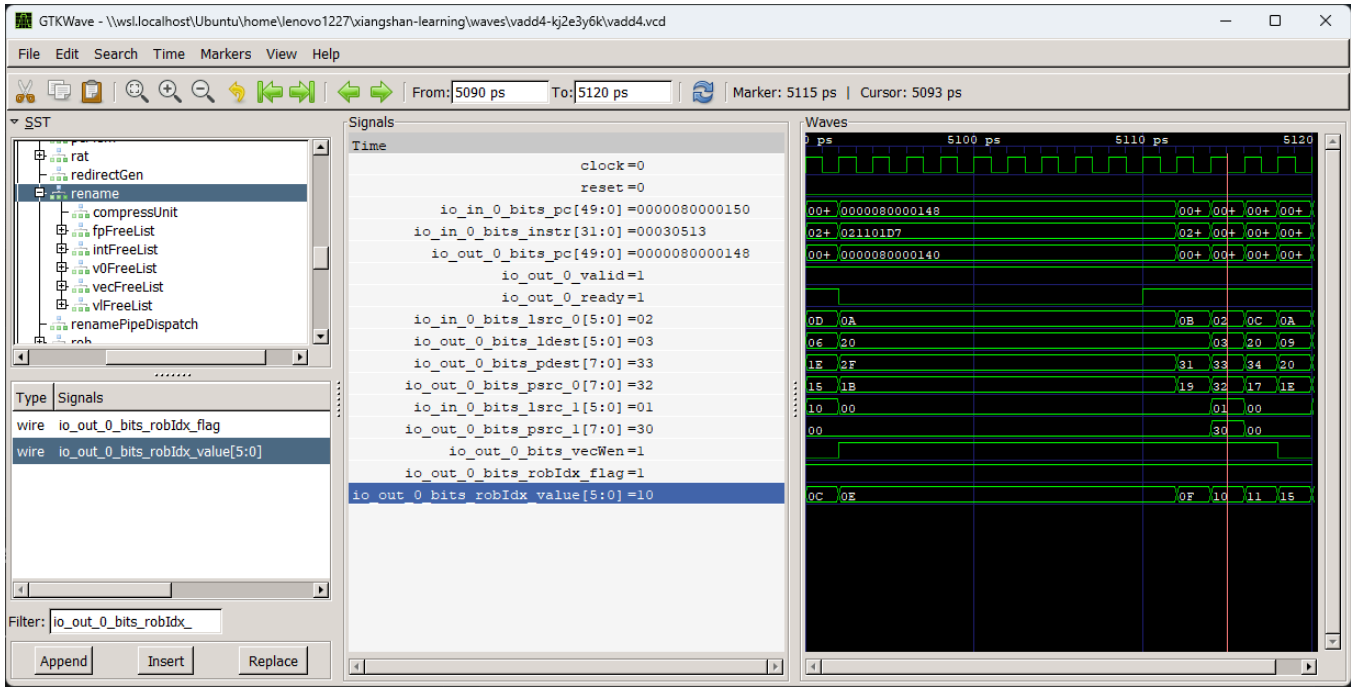


图 3-10　重命名后的物理源、物理目的和 ROB 身份，为后端追踪提供共同标识。

分配物理编号是为了区分同一逻辑寄存器的不同版本；源数据何时可读，还取决于写出这些物理寄存器的前序指令何时完成。5117 ps 的派发接口虽然 valid/ready 为 1/1，两个 `srcState` 仍为 0，因此目标在 5118 ps 入队后等待源数据。

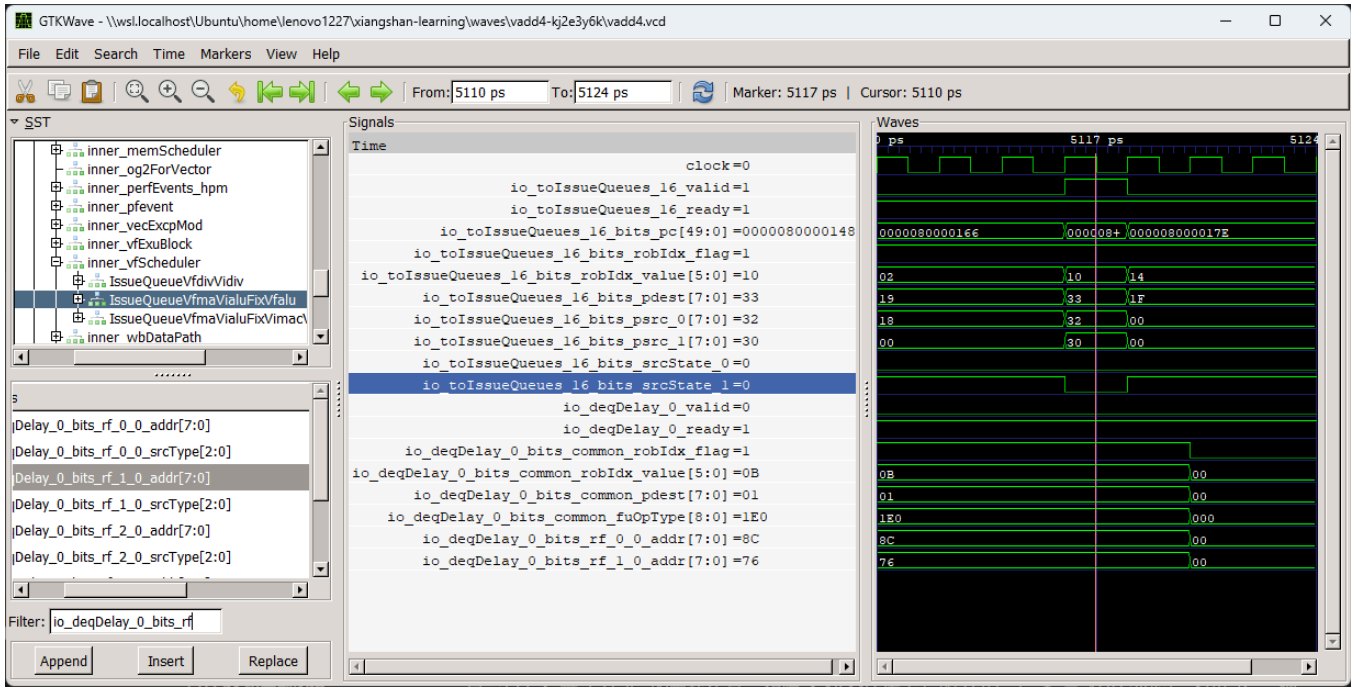


图 3-11　目标进入发射队列时两个源尚未就绪。派发 ready 表示发射队列还能接收新条目；操作数是否就绪由 srcState 另行表示。

3.4.3 写回唤醒与发射

5251 ps 的有效写回标签为 48，使源 1 在 5252 ps 就绪；5257 ps 的标签为 50，使源 0 在 5258 ps 就绪。到 5259 ps，条目的 `canIssue=1`。队列随后选择该条目，5261 ps 的发射接口 valid/ready 为 1/1，携带 ROB `1/16`、pdest 51 和两个物理读地址 50、48，于 5262 ps 接受。

由此可以解释表 3-1 中较长的等待区间：这次动态执行的大部分时间花在等待源就绪和调度入发射上。因此，指令端到端周期与执行单元计算延迟需要分开看。

3.5 计算结果怎样成为架构状态

3.5.1 实际操作数与计算完成

5267 ps，父执行单元 EXU2 的入口握手有效，ROB/pdest 与此前相同。两个 128 位源按 32 位分组为：

```
src0 = 00000028_0000001e_00000014_0000000a
src1 = 00000004_00000003_00000002_00000001
VL   = 4
```

右端为元素 0，因此它们分别是 B 和 A。5268 ps 接受输入后，5269 ps 的 `VialuFix` 输出已经携带完整和，输出 valid/ready 为 1/1，5270 ps 完成握手。

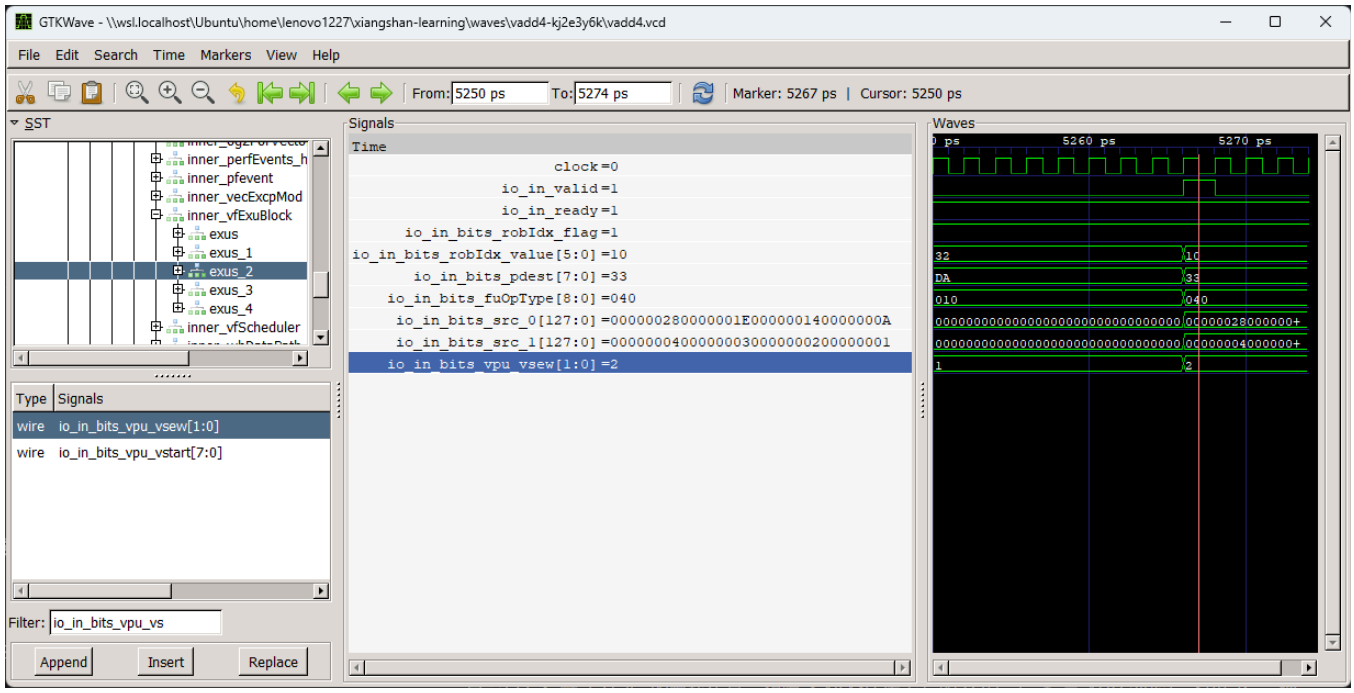


图 3-12　5268 ps 接受的操作数为两组四元素向量，ROB `1/16` 和 pdest 51 与重命名一致。

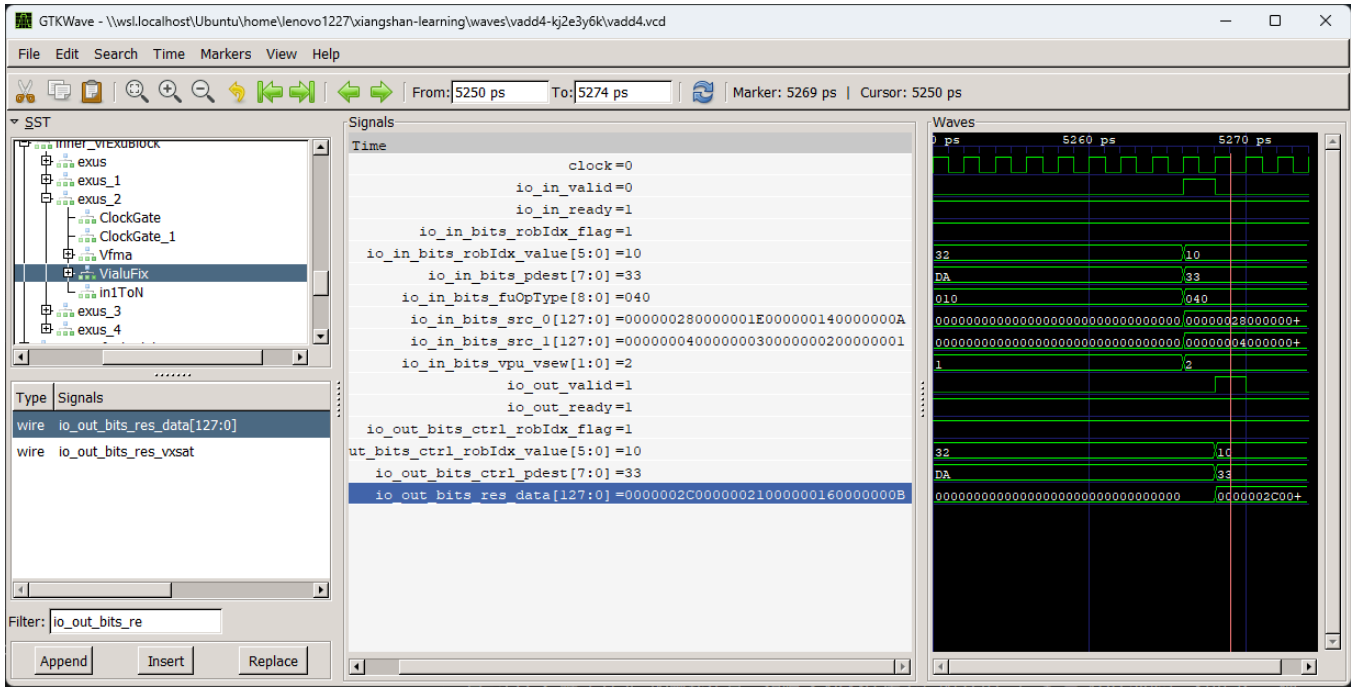


图 3-13　5270 ps 输出的 128 位结果为 `0x0000002c00000021000000160000000b`，对应 `[11,22,33,44]`。

3.5.2 物理写入、退休和映射更新

运算输出经写回数据通路送到向量寄存器堆。5271 ps 写口 `wen=1`、地址为 51，5272 ps 四个 32 位分片的 `mem_51` 分别写入 `0x0b`、`0x16`、`0x21`、`0x2c`。此时结果已经存在于物理寄存器中。

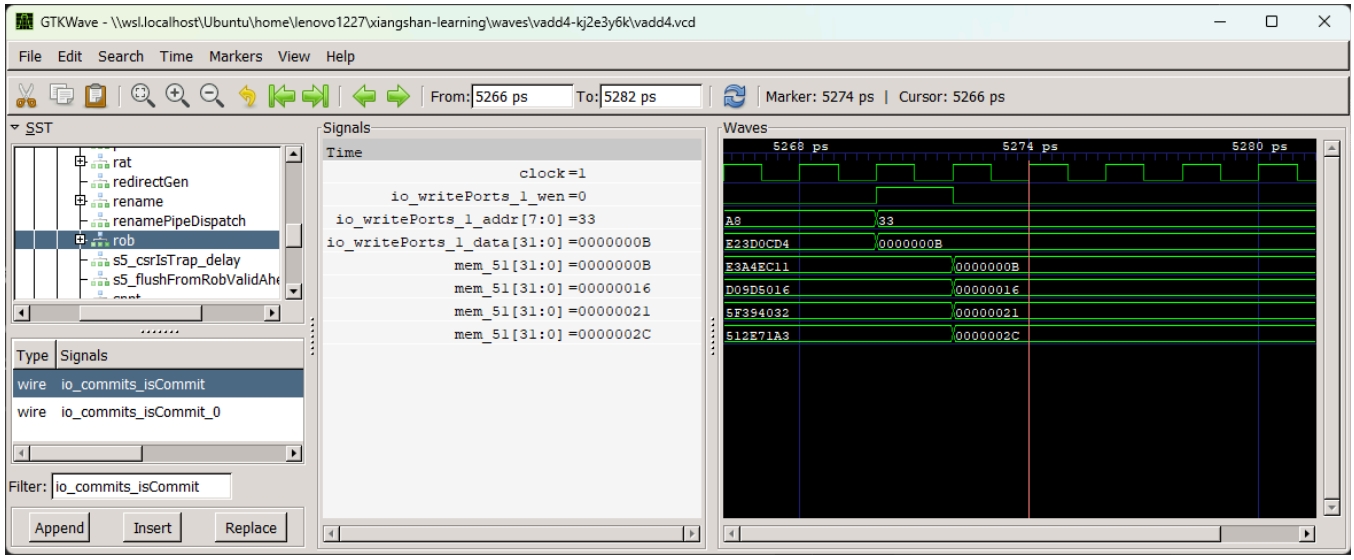


图 3-14　物理写入发生于 5272 ps，图中 Marker=5274 ps 时各分片保持新值。

随后，5273 ps 的 ROB 退休槽 0 满足 `isCommit=commitValid_0=1`，PC、机器码、ROB `1/16` 和 pdest 51 均匹配目标，指令在 5274 ps 按序退休。这一步确认该结果已按程序次序提交。

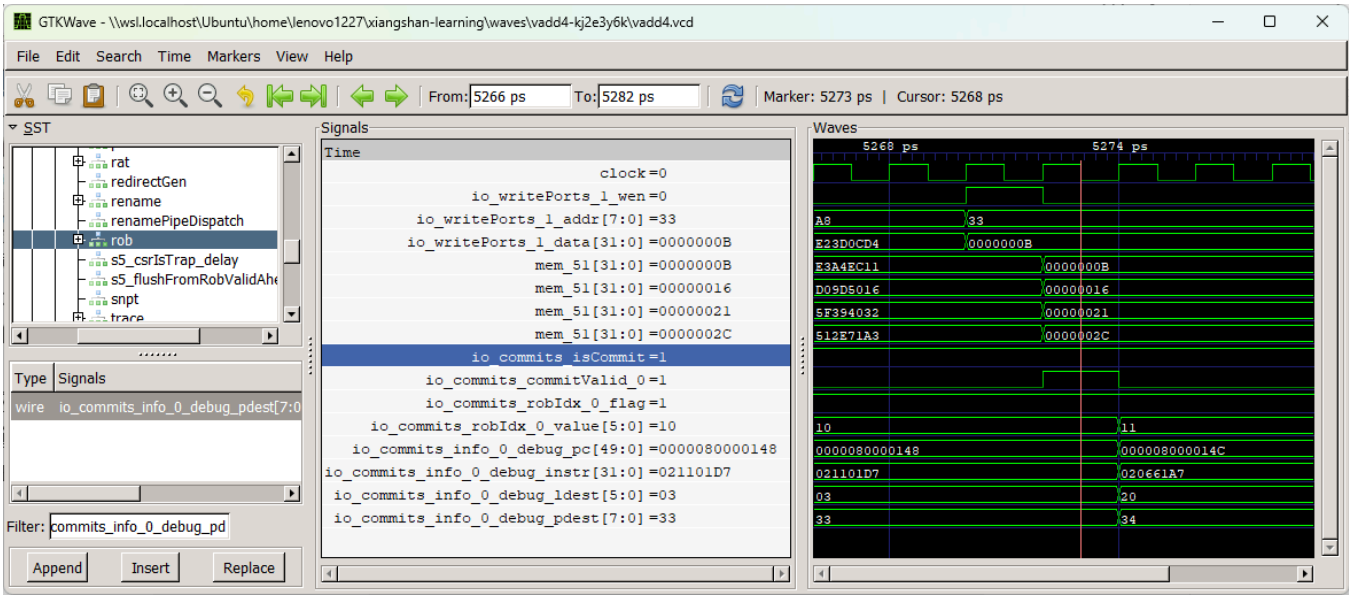


图 3-15　5273 ps 的有效退休槽确认 5274 ps 退休事件。

退休信息再经 RAB 更新向量已提交映射。5277 ps，映射写口准备把逻辑 v3 指向物理 51，`arch_table_3` 尚为旧编号 3；5278 ps 沿后，表项变为 51。

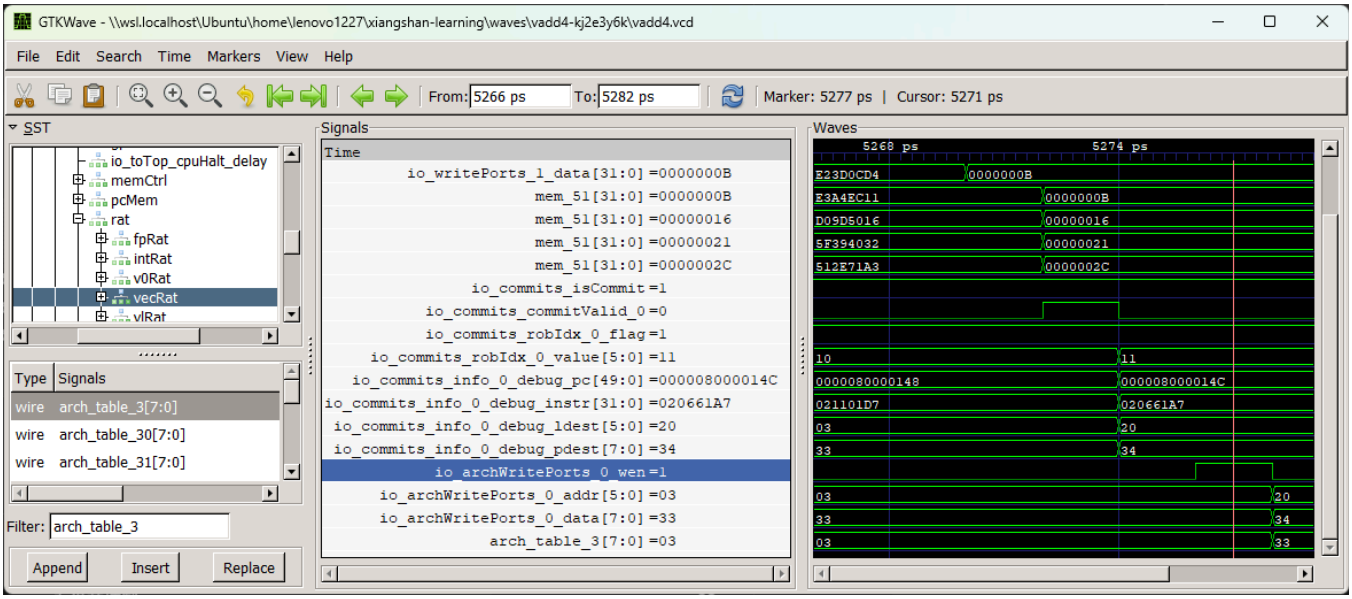


图 3-16　5277 ps 的写口为 addr=3、data=51，已提交表项仍为旧编号 3。

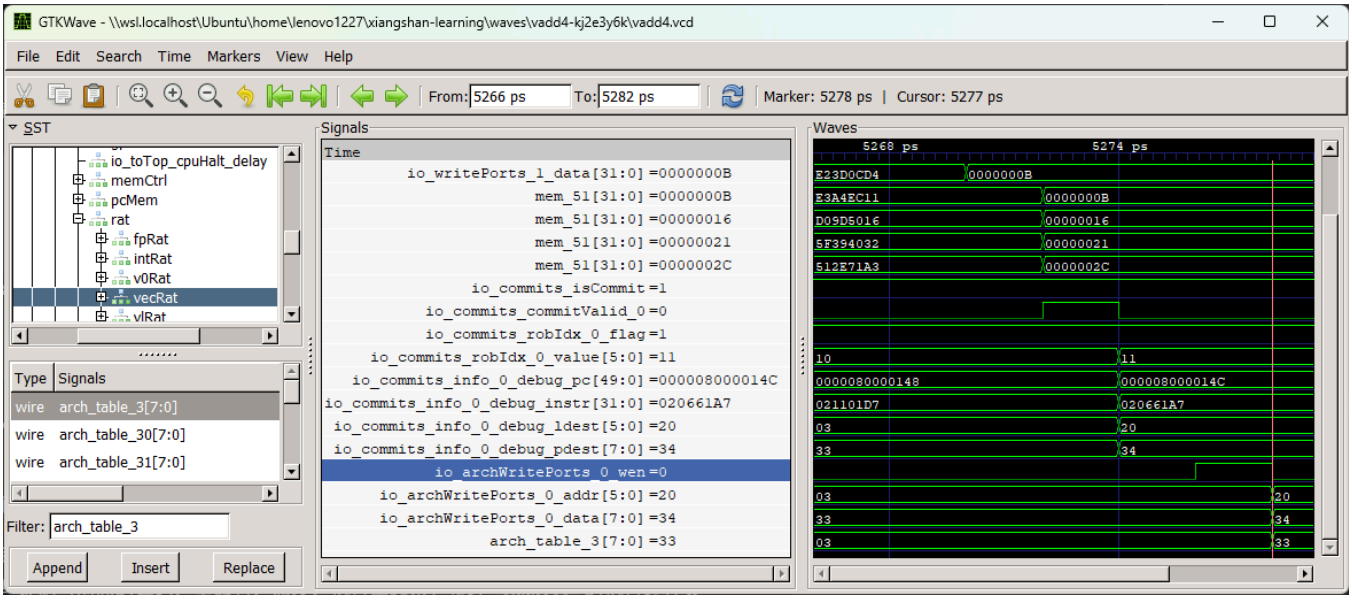


图 3-17　5278 ps 后 v3 的已提交映射指向物理寄存器 51。

这三个时刻承担不同职责：5272 ps 保存计算数据，5274 ps 确认按序退休，5278 ps 更新已提交的寄存器版本。映射表保存的是物理编号，128 位运算结果早已写入对应寄存器。把这三步分开观察后，就能从 FTQ 取指请求、经 ROB 与物理寄存器编号，一直追到最终的已提交映射，说明这一次 VADD 如何完成。

本章观察的是指定 e32/m1/VL4 实例。观测从前端 FTQ 请求起，经后端源依赖等待，到提交更新为止；缓存缺失处理、BPU 内部预测及其他向量配置未纳入本次观测。复现材料与完整信号索引集中于阶段一附录。

附录 A 环境与波形索引

本附录用于恢复第一阶段所用的源码与二进制版本，并定位完整原始记录。正文已给出关键构建、运行命令与波形分析。下列哈希来自保存的验收记录，未在本次文档重写中重新运行硬件实验。

A.1 源码与二进制身份

表 A-1　源码版本。

对象	提交
XiangShan	16ae9ddcda54fc9a2fddffad73174cf793ac7814
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
difftest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
ready-to-run	ed8c30e1ec8a9c9aa6b116ce54281235d00273fa

表 A-2　第一阶段二进制文件 SHA256。

文件	保存的 SHA256
emu	9ee6ce3e8372c6345fc7219563c5ac63c9b3517adb87dc347816d14f228fd7e2
预编译参考库	7af2120acefa83b7299c91b99132f4087dd8ab0f7219461b9aea48ce38891e03
Hello BIN	195dca0c3a748fea70efa84e1d63cd3926098189f61fc6babe80214425b1f479
VADD BIN	0fa5845070b3f954cf375cf571692326971cda6a994d1f20ee65a626d4357283

恢复时按第 2 章准备工具和源码，分别构建 emu 与 AM 应用；按第 3 章导出新 VCD，并保留运行日志与退出状态。以上校验值用于定位历史实验所用的二进制文件；在新环境重新编译后，另行记录文件校验值。

A.2 完整运行和事件记录

内容	工程入口
环境与版本核对	验收记录〔5〕
Hello 完整运行包	源码、镜像、日志与身份〔7〕
VADD 应用构建	编译文件清单〔8〕
VADD 导波形	运行及退出状态〔9〕、波形清单〔10〕
VADD 汇编	源程序〔11〕
前端逐时刻观测	前端记录目录〔12〕
执行阶段观测	操作数与完成事件〔13〕
写回与退休观测	物理写入、退休及映射〔14〕

A.3 波形信号与源码导航

以下层级位于 `TOP.SimTop.l_soc.core_with_l2.core` 下，源码相对锁定版本的 `src/main/scala/xiangshan/`。这些名称用于在本次 VCD 中定位信号；事件是否成立，仍以正文给出的 valid/ready 等判定条件为准。

表 A-3　前端定位。

阶段	波形层级及主要信号	源码
FTQ	`frontend.inner_ftq`；`io_toIfu_req_valid/ready`、startAddr、ftqIdx	`frontend/FrontendBundle.scala`、`Frontend.scala`
ICache	`frontend.inner_icache`；`io_fetch_resp_valid`、vaddr、doubleline、data	`frontend/icache/ICacheMainPipe.scala`
IFU	`frontend.inner_ifu`；f2 FTQ、`f2_fire`、`f2_cut_ptr_10/11`、toIbuffer 位置10	`frontend/IFU.scala`
IBuffer	`frontend.inner_ibuffer`；`io_out_5_*`、decodeCanAccept、flush	`frontend/IBuffer.scala`
后端入口	`backend.inner_ctrlBlock`；cfVec5、decodeBuf 槽3/1/0	`backend/CtrlBlock.scala`
译码入口	`backend.inner_ctrlBlock.decode`；in0 valid/ready、PC、指令字	`backend/decode/DecodeUnit.scala`

表 A-4　后端定位，层级在 `backend` 下。

阶段	波形层级及主要信号	源码
重命名	`inner_ctrlBlock.rename`；out0 逻辑/物理编号、ROB	`backend/rename/Rename.scala`
派发	`inner_ctrlBlock.dispatch`；`io_toIssueQueues_16_*`	后端派发逻辑
唤醒发射	`inner_vfScheduler.IssueQueueVfmaVialuFixVfalu`；wakeUpFromWB4、srcStatus、deqDelay0	`backend/issue/EntryBundles.scala`、`IssueQueue.scala`
执行	`inner_vfExuBlock.exus_2`、`VialuFix`；src、res.data	`backend/fu/vector/VecPipedFuncUnit.scala`、`backend/fu/wrapper/VIAluFix.scala`
物理写入	`inner_dataPath.vfRegFilePart0`～`Part3`；writePorts1、mem51	`backend/datapath/DataPath.scala`、`backend/regfile/Regfile.scala`
退休	`inner_ctrlBlock.rob`；isCommit、commitValid0、身份	`backend/rob/Rob.scala`
已提交映射	`inner_ctrlBlock.rat.vecRat`；archWritePorts0、arch_table3	`backend/rename/RenameTable.scala`

本例的 PC、ROB/pdest 和时标见第 3 章。其他运行需按其自身记录重新核对配置、信号层级与指令标识。

材料完整路径

编号与本阶段正文一致。路径相对于配套 RISCV_vdot 工程根目录；图片随本文档目录提供，完整源码、日志和大型 VCD 由配套工程提供。

编号	材料	项目相对路径
1	部署记录	`artifacts/deployment-history-2026-10-19/build-retry.log`
2	构建时间记录	`artifacts/stage1-environment-review-2026-10-19/history/build-time.log`
3	Hello 原始日志	`artifacts/runs/S1-01/hello-meijie-20261019-120654-8FtdkQ/run.log`

编号	材料	项目相对路径
4	退出记录	artifacts/runs/S1-01/hello-meijie-20261019-120654-8FtdkQ/run-result.txt
5	环境核对记录	artifacts/stage1-environment-review-2026-10-19/evidence-review.json
6	原波形指南	exercises/vadd4/WAVE.md
7	源码、镜像、日志与身份	artifacts/runs/S1-01/hello-meijie-20261019-120654-8FtdkQ
8	编译文件清单	artifacts/vadd4-2026-10-17/manifest.json
9	运行及退出状态	artifacts/vadd4-wave-2026-10-17/run-result.json
10	波形清单	artifacts/vadd4-wave-2026-10-17/manifest.json
11	源程序	exercises/vadd4/vadd4.S
12	前端记录目录	artifacts/vadd4-frontend-2026-10-19
13	操作数与完成事件	artifacts/vadd4-execution-2026-10-18/observations.json
14	物理写入、退休及映射	artifacts/vadd4-writeback-2026-10-18/observations.json